Q1. Consider the relations:

- **SAILORS(SID, SNAME, RATING, AGE)**
- **BOATS(BID, BNAME, COLOR)**
- **RESERVES(SID, BID, DAY)**

a) Find the average age of all sailors

b) Display the number of sailors for each rating level

c) Display the number of reservations made for each boat.

d) Find the names of sailors who have reserved boat number 104 using IN operator

e) Find the names of sailors who have reserved all boats using nested queries.

Q2. Consider the relations:

- **SAILORS(SID, SNAME, RATING, AGE)**
- **BOATS(BID, BNAME, COLOR)**
- **RESERVES(SID, BID, DAY)**

a) Create a view named HIGH_RATING_SAILORS to display sailors whose rating is > 7.

b) Create a function that accepts a sailor's SID and returns the sailor's name.

c) Find the names of sailors who have reserved all boats.

d) Write a PL/SQL procedure to display the day of the week for a given date.

e) Write a BEFORE INSERT trigger to prevent insertion of duplicate SID values.

Q3. Consider the relations:

- **SAILORS(SID, SNAME, RATING, AGE)**
- **BOATS(BID, BNAME, COLOR)**
- **RESERVES(SID, BID, DAY)**

a) Write a correlated nested query to find the names of sailors who have reserved boat number 103.

b) Find the names of sailors who have reserved a RED boat but have not reserved a GREEN boat.

c) Find the maximum age of sailors for each rating level.

d) Display rating levels whose average age is greater than 30.

e) Find sailors whose rating is greater than the rating of all sailors named Horatio.

Q4. Consider the relations:

- **SAILORS(SID, SNAME, RATING, AGE)**
- **BOATS(BID, BNAME, COLOR)**
- **RESERVES(SID, BID, DAY)**

a) Create a view named SAILOR_DETAILS containing SID, SNAME and AGE.

b) Create a function that accepts a sailor's SID and returns the sailor's age.

c) Write a BEFORE INSERT trigger to prevent insertion of sailors whose age is less than 18.

d) Write a PL/SQL program using an implicit cursor to update the rating of all sailors and display the number of records updated using SQL%ROWCOUNT.

e) Write a PL/SQL procedure to display the complete details of a sailor for a given SID.

Q5. Consider the relations:

- **EMP(EMPNO, ENAME, JOB, SAL, DEPTNO)**
- **DEPT(DEPTNO, DNAME, LOC)**

a) Display employees working as MANAGER

b) Display employee names along with their department numbers

c) Find the average salary of all employees.

d) Display department numbers having more than two employees using GROUP BY and HAVING.

e) Create a view named HIGH_SALARY_EMP to display employees whose salary is greater than 3000.

Q6. Consider the relations:

- **EMP(EMPNO, ENAME, JOB, SAL, DEPTNO)**
- **DEPT(DEPTNO, DNAME, LOC)**

a) Create a function that accepts an employee number and returns the employee's salary.

b) Create a BEFORE INSERT trigger to prevent insertion of an employee whose salary is less than 500.

c) Find the highest-paid employee in each department using a correlated nested query.

d) Create a PL/SQL package named HR_PACKAGE containing the following procedures:

- HIRE procedure to insert a new employee record into the EMP table.
- FIRE procedure to delete an employee record based on EMPNO.

e) Write a PL/SQL program using an explicit cursor to display employee names and salaries.

Q7. Consider the relations:

- **EMP(EMPNO, ENAME, JOB, SAL, DEPTNO)**
- **DEPT(DEPTNO, DNAME, LOC)**

a) Count the number of employees in the EMP table

b) Update the salary of employee number 1001 by 10%.

c) Create a view named EMP_BASIC_DETAILS to display EMPNO, ENAME and JOB.

d) Display the average salary of employees for each department

e) Display employees whose job contains the word 'MAN'.

Q8. Consider the relations:

- **EMP(EMPNO, ENAME, JOB, SAL, DEPTNO)**
- **DEPT(DEPTNO, DNAME, LOC)**

a) Declare a procedure to display employee details for a given employee number.

b) Write PL/SQL functions to return the department name for a given department number.

c) Write PL/SQL procedures to accept an employee number and return the annual salary.

d) Write BEFORE INSERT triggers to prevent inserting an employee with a salary less than 1000.

e) Write PL/SQL programs using parameterized cursors to display employees belonging to a given department number.